南　　开　　大　　学

密 码 与 网 络 空 间 安 全 学 院

**网络安全技术**

---

# Lab2：基于 RSA 算法自动分配密钥的加密聊天程序

---


肖俊涛

年级：2023 级

专业：信息安全与法学双学位班

指导教师：张玉


2026 年 4 月 27 日

# 目录

**一、 实验要求** **1**

**二、 实验环境** **2**

（一） 软硬件环境 . . . . . 2

（二） 工程结构 . . . . . 2

（三） 编译与运行方式 . . . . . 2

（四） 命令行操作与参数说明 . . . . . 3

**三、 实验原理** **5**

（一） RSA 算法与密钥生成 . . . . . 5

（二） 混合加密设计与 DES 会话密钥自动分配 . . . . . 5

（三） TCP 协议与可靠传输机制 . . . . . 6

（四） Socket 编程与程序工作流程 . . . . . 7

**四、 实验核心代码解析** **9**

**五、 实验结果和分析** **13**

**六、 收获与反思** **15**

# 一、 实验要求

本次实验在第一次实验“基于 DES 加密的 TCP 聊天程序”的基础上继续扩展，目标是实现一个**基于 RSA 自动分配 DES 会话密钥的加密聊天程序**。实验要求可概括如下。

1. 在 Linux 环境下完成客户端与服务器端的 TCP 全双工聊天程序，保留第一次实验中已经实现的 DES 加密通信能力。
2. 新增 RSA 非对称加密模块，由服务器端自动生成 RSA 公钥/私钥对，并通过已建立的 TCP 连接将公钥发送给客户端。
3. 客户端自动生成 8 字节 DES 会话密钥，使用服务器端公钥进行加密，并将加密后的结果发送给服务器端，实现 DES 密钥的自动分发。
4. 在密钥分发完成后，通信双方自动切换到 DES 加密聊天阶段，要求加密与解密过程对用户透明，用户只需输入和查看聊天内容。
5. 程序应支持双向通信、命令行参数配置、密文显示选项以及本地退出控制等交互功能，并保持工程结构清晰、可编译、可运行。

# 二、 实验环境

## (一) 软硬件环境

| 项目 | 说明 |
|---|---|
| 宿主系统 | Windows 平台 |
| 实验运行环境 | WSL2 下的 `Ubuntu-OSLab-Recovered` |
| 编译器 | `g++ 13.3.0`（支持 C++17） |
| 构建工具 | `CMake 3.28.3`（工程最低要求为 3.16） |
| 主要语言 | C++17 |
| 网络编程接口 | Linux Socket API |
| 线程支持 | `std::thread`、`std::atomic` |
| 核心密码模块 | 手写 DES 模块 + 手写 RSA 模块 |

## (二) 工程结构

本实验仍采用多文件工程结构，在第一次实验的基础上新增 RSA 模块，并保留 DES 加密聊天与 Socket 通信模块。当前工程目录结构如下。

```
Lab02
    CMakeLists.txt
    README.md
    main.cpp
    include
        DES_Operation.h
        RSA_Operation.h
        chat.h
    src
        DES_Operation.cpp
        RSA_Operation.cpp
        chat.cpp
```

## (三) 编译与运行方式

本实验采用 CMake 构建工程，可执行文件名为 `RSA_DES_chat`。核心编译命令如下。

```
wsl -d Ubuntu-OSLab-Recovered
cd /mnt/c/Users/xjt26/Desktop/2313226肖俊涛网络安全技术lab2
cmake -S . -B build
cmake --build build -j
```

编译后可执行文件输出到 `bin` 目录中。程序分别以服务器端和客户端模式运行：

```
./bin/RSA_DES_chat server --port 8888 --show-cipher yes
./bin/RSA_DES_chat client --ip 127.0.0.1 --port 8888 --show-cipher yes
```

与第一次实验不同，当前程序**不再要求用户手动输入 DES 密钥**。DES 会话密钥在连接建立后由客户端自动生成，并借助 RSA 公钥加密后发送给服务器端。

## （四） 命令行操作与参数说明

由于本实验程序在 Linux 命令行环境下运行，因此需要对实际操作命令和参数含义进行统一说明。实验中使用的典型命令序列如下。

### (1) 进入 Linux 环境并编译工程

```
wsl -d Ubuntu-OSLab-Recovered
cd /mnt/c/Users/xjt26/Desktop/2313226肖俊涛网络安全技术lab2
cmake -S . -B build
cmake --build build -j
```

上述命令依次完成以下工作：进入 WSL 中的 Ubuntu 发行版、切换到实验目录、生成构建目录并编译当前工程。编译完成后，可执行文件输出到 `bin` 目录中。

### (2) 服务端与客户端启动命令

本实验需要打开两个终端，分别启动服务端与客户端。服务端命令如下：

```
./bin/RSA_DES_chat server --port 8888 --show-cipher yes
```

客户端命令如下：

```
./bin/RSA_DES_chat client --ip 127.0.0.1 --port 8888 --show-cipher yes
```

若希望在聊天过程中显示十六进制密文，则可以额外加入 `--show-cipher yes`。若命令中省略该参数，程序会在建立聊天前通过命令行询问用户是否显示密文。

### (3) 运行参数含义

程序运行时涉及的主要参数如表 1 所示。

表 1: 程序运行命令及参数含义

| 参数或命令 | 含义说明 |
|---|---|
| `server` | 指定程序以服务器端模式运行，负责监听端口并等待客户端连接 |
| `client` | 指定程序以客户端模式运行，负责主动连接服务器 |
| `--ip 127.0.0.1` | 指定客户端连接的目标 IP 地址；`127.0.0.1` 表示本机回环地址，适用于在同一台主机上进行测试 |
| `--port 8888` | 指定通信端口号；服务端监听该端口，客户端也必须连接同一端口 |
| `--show-cipher yes/no` | 指定是否在终端中显示十六进制密文；用于观察加密结果和展示实验效果 |
| `quit` | 本地退出控制命令；输入后程序直接关闭本端连接，而不会将其作为普通聊天消息发送 |

**(4) 命令行交互在本实验中的作用**

在本实验中,命令行参数主要承担角色切换、网络配置和演示配置三个作用。通过 `server/client` 参数可以在同一份可执行程序中切换不同身份；通过 `--ip` 和 `--port` 参数可以指定连接目标；通过 `--show-cipher` 参数可以选择是否显示十六进制密文。由于 DES 会话密钥已经改为自动分发，因此用户层面不再需要手动输入密钥，程序使用方式也更加直观。

# 三、 实验原理

## (一) RSA 算法与密钥生成

RSA 是典型的非对称密码算法，其核心思想是使用一组不同的密钥分别完成加密和解密：公钥可以公开发布，私钥由接收方保存。本实验中，服务器端负责生成 RSA 密钥对并分发公钥，客户端利用该公钥安全传输后续聊天所需的 DES 会话密钥。

RSA 的基本构造如下。首先随机选择两个大素数 $p$ 与 $q$，计算模数

$$n = pq, \tag{1}$$

再计算欧拉函数

$$\varphi(n) = (p-1)(q-1). \tag{2}$$

随后选择公钥指数 $e$，要求满足

$$1 < e < \varphi(n), \qquad \gcd(e, \varphi(n)) = 1. \tag{3}$$

再求出私钥指数 $d$，使其满足

$$ed \equiv 1 \pmod{\varphi(n)}. \tag{4}$$

因此，RSA 公钥和私钥分别写作

$$\text{PublicKey} = (e, n), \qquad \text{PrivateKey} = (d, n). \tag{5}$$

对于明文块 $m$，加密与解密公式分别为

$$c \equiv m^e \pmod{n}, \tag{6}$$

$$m \equiv c^d \pmod{n}. \tag{7}$$

其正确性来源于模逆关系和欧拉定理。由于 $ed \equiv 1 \pmod{\varphi(n)}$，可写为 $ed = 1 + k\varphi(n)$，从而在满足条件时有

$$m^{ed} = m^{1+k\varphi(n)} \equiv m \pmod{n}, \tag{8}$$

因此解密能够恢复原始明文。

在本实验的具体实现中，RSA 模块从预先构造的素数表中随机选取 $p$ 与 $q$，优先使用常见公钥指数 65537；若其与 $\varphi(n)$ 不互素，则自动搜索其他奇数指数。私钥指数 $d$ 通过扩展欧几里得算法求逆得到。为了适配课程实验规模，程序采用**逐字节加密**方式处理 DES 会话密钥中的每个字符，因此要求每个明文字节都严格小于模数 $n$，这一条件在当前选取的素数范围内可以满足。

## (二) 混合加密设计与 DES 会话密钥自动分配

本实验并没有直接使用 RSA 加密全部聊天内容，而是采用 **RSA + DES 的混合加密结构**。这是因为 RSA 适合处理短数据和密钥分发，而 DES 属于对称分组密码，更适合高频率、连续性的聊天数据加密。

具体地，程序将通信过程划分为两个阶段：

1. **密钥分发阶段：**服务器端生成 RSA 公钥/私钥对并发送公钥；客户端自动生成 8 字节 DES 会话密钥，用 RSA 公钥加密后发送给服务器端；服务器端使用私钥解密并恢复该会话密钥。

2. **安全聊天阶段：**双方将会话密钥写入 DES 模块，对用户输入的消息进行分组加密和解密，完成后续的透明聊天。

这一设计相较于第一次实验的“手工预共享密钥”方式有两个直接改进：其一，用户不再需要手动输入密钥，交互更自然；其二，DES 密钥不会以明文形式出现在命令行中，密钥传递过程更加符合保密通信的实验目标。

图 1 展示了本实验中自动分配会话密钥的总体流程。

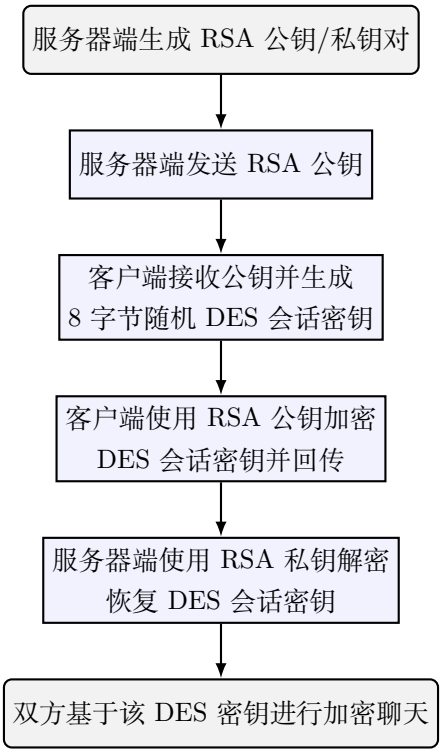



图 1: 基于 RSA 的 DES 会话密钥自动分配流程

在会话密钥生成方面，程序从大小写字母和数字构成的字符集内随机选取 8 个字符，形成长度固定的 DES 密钥。由于第一次实验已经实现 DES 模块，因此 Lab2 的核心在于：**用 RSA 安全地把这把 DES 密钥送到服务器端**，然后继续复用已完成的 DES 聊天部分。

## (三) TCP 协议与可靠传输机制

TCP 是面向连接的传输层协议，具有可靠传输、按序到达和全双工通信等特性。对于聊天程序而言，这些特性能够保证双方在同一条连接上稳定地双向交换数据，因此仍然适合作为本实验的底层通信协议。

在 Lab2 中，TCP 的作用不仅体现在聊天阶段，也体现在前置的密钥分发阶段。服务器端先监听端口，客户端再主动发起连接；连接建立后，同一条 TCP 连接上依次传输 RSA 公钥、加密后的 DES 会话密钥以及后续 DES 密文消息。这说明 TCP 在本实验中承担的是**统一的数据承载通道**作用。

需要注意的是，TCP 面向应用层提供的是连续字节流，而不是天然带边界的报文。因此，如果简单连续发送多类数据，接收端无法仅凭一次 `recv` 操作判断当前读到的是哪一类消息、长度是多少。为了解决这一问题，本实验在应用层设计了统一的**分帧机制**：每个报文由“1 字节类型字段 + 4 字节长度字段 + 有效载荷”组成。具体类型定义如表 2 所示。

表 2: Lab2 中的应用层报文类型

| 报文类型 | 类型值 | 作用说明 |
|---|---|---|
| kRsaPublicKey | 1 | 服务器端向客户端发送 RSA 公钥 |
| kEncryptedDesKey | 2 | 客户端向服务器端发送经 RSA 加密后的 DES 会话密钥 |
| kChatCipherText | 3 | 双方在聊天阶段发送的 DES 密文消息 |

这一分帧设计的实际意义在于：

1. 使密钥分发报文与聊天密文能够在同一条 TCP 连接上被正确区分；
2. 解决 TCP 字节流带来的粘包与拆包问题；
3. 便于接收端先判断报文类型，再按长度读取完整载荷；
4. 为后续进一步扩展更多控制报文预留接口。

## （四） Socket 编程与程序工作流程

Socket 可以理解为网络通信端点。Linux 下的 TCP 编程本质上是围绕套接字文件描述符完成“建连、收发、关闭”全过程。本实验在第一次实验的基础上保留了 Socket 编程框架，同时在连接建立后插入 RSA 密钥分发逻辑。

### (1) 主要接口及其作用

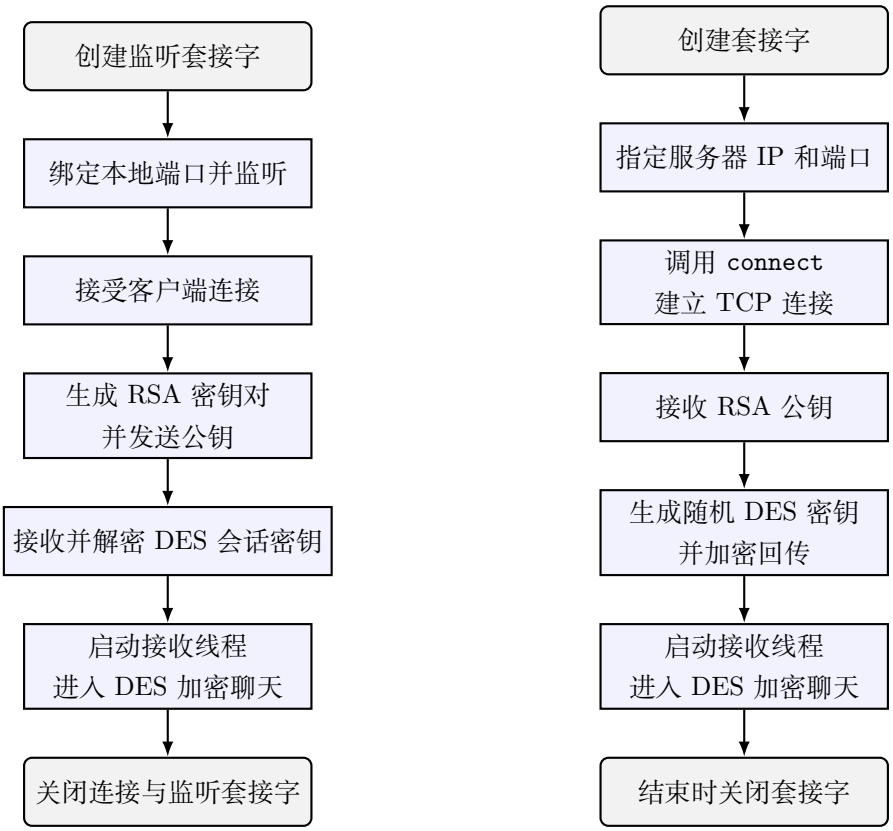
本实验使用的主要 Socket 接口如表 3 所示。

表 3: TCP Socket 主要接口及作用

| 接口 | 作用说明 |
|---|---|
| socket | 创建套接字，指定地址族 AF_INET 和类型 SOCK_STREAM |
| bind | 将服务器端监听套接字绑定到本地端口 |
| listen | 使服务器端进入监听状态，等待客户端连接 |
| accept | 取出一个连接请求并返回新的已连接套接字 |
| connect | 客户端主动向目标地址和端口发起连接 |
| send | 在已连接套接字上发送报文数据 |
| recv | 在已连接套接字上接收报文数据 |
| shutdown/close | 关闭通信连接并释放套接字资源 |

### (2) 服务器端与客户端的工作流程

如图 2 所示，Lab2 中服务器端与客户端在建立连接后会先完成 RSA 握手，再进入 DES 加密聊天阶段。




（a）服务器端工作流程 （b）客户端工作流程

图 2: Lab2 中服务器端与客户端的工作流程

**(3) 并发收发与透明聊天**

为了保证用户在输入消息的同时仍能接收对方消息，程序采用主线程负责发送、接收线程负责异步接收的并发模型。密钥协商完成后，双方都会启动接收线程；主线程读取终端输入并在本地判断是否为 `quit`，若为普通消息则先执行 DES 加密，再将密文封装后发送。接收线程则持续读取完整报文，解密后将明文显示到终端。

这种设计使用户几乎感知不到 RSA 分发密钥和 DES 加密传输的底层细节。对于使用者而言，程序表现为一个普通聊天窗口；对于实现而言，其内部已经完成了建连、密钥传输、会话加密和退出控制等完整过程。

# 四、 实验核心代码解析

## 1. 命令行参数与程序入口

程序入口位于 `main.cpp`,负责区分服务端和客户端模式,并解析 `--ip`、`--port`、`--show-cipher` 等参数。与第一次实验相比，Lab2 已经取消了 `--key` 参数，因为 DES 会话密钥将在握手阶段自动生成并分发。

```
if (mode == "server" || mode == "s") {
    config.isServer = true;
} else if (mode == "client" || mode == "c") {
    config.isServer = false;
}

if (showCipherOption.has_value()) {
    std::cout << "Show cipher text: "
              << (config.showCipherHex ? "yes" : "no") << '\n';
} else {
    config.showCipherHex = AskShowCipher();
}
```

这一设计的优点在于：

- 运行方式直观，便于实验演示；
- 服务器端和客户端共用同一份可执行程序；
- 密文显示选项既可通过命令行指定，也可交互选择；
- 用户不再需要手动输入 DES 密钥。

## 2. RSA 密钥对生成与 DES 会话密钥生成

Lab2 的第一个新增模块是 RSA 算法实现，对应文件为 `RSA_Operation.cpp`。程序首先构造素数表，再随机选取两个不同素数生成密钥对，随后为会话通信准备随机 DES 密钥。

```
void RsaOperation::GenerateKeyPair() {
    const auto primes = BuildPrimeTable();
    ...
    const std::uint64_t p = primes[dist(generator)];
    std::uint64_t q = primes[dist(generator)];
    ...
    const std::uint64_t n = p * q;
    const std::uint64_t phi = (p - 1) * (q - 1);

    std::uint64_t e = 65537;
    if (Gcd(e, phi) != 1) {
        e = 3;
        while (e < phi && Gcd(e, phi) != 1) {
            e += 2;
        }
```

```
    }

    const std::uint64_t d = ModInverse(e, phi);
    publicKey_ = {e, n};
    privateKey_ = {d, n};
}

std::string RsaOperation::GenerateRandomDesKey() {
    std::string key;
    key.reserve(8);
    for (int i = 0; i < 8; ++i) {
        key.push_back(kDesAlphabet[dist(generator)]);
    }
    return key;
}
```

这部分代码完成了 Lab2 的两个关键准备：

1. 服务器端具备了自动生成 RSA 公钥/私钥对的能力；
2. 客户端具备了自动生成 8 字节 DES 会话密钥的能力。

相较于 Lab1，程序不再依赖人工约定密钥，而是将密钥生成过程纳入到程序逻辑内部。

### 3. RSA 握手与密钥自动分发

真正体现 Lab2 升级核心的是 TCP 握手过程。服务器端先发送 RSA 公钥，客户端接收公钥后生成随机 DES 密钥，并使用 RSA 公钥加密后回传；服务器端再用私钥解密并写入 DES 模块。

```
bool Chat::PerformServerHandshake() {
    rsa_.GenerateKeyPair();
    SendFrame(PacketType::kRsaPublicKey,
              SerializePublicKey(rsa_.GetPublicKey()));

    PacketType type = PacketType::kRsaPublicKey;
    std::vector<std::uint8_t> payload;
    ReceiveFrame(type, payload);
    ...
    sessionKey_ = RsaOperation::DecryptText(encryptedKey,
                                            rsa_.GetPrivateKey());
    des_.SetKey(sessionKey_);
    return true;
}

bool Chat::PerformClientHandshake() {
    PacketType type = PacketType::kRsaPublicKey;
    std::vector<std::uint8_t> payload;
    ReceiveFrame(type, payload);
    ...
```

```
    sessionKey_ = RsaOperation::GenerateRandomDesKey();
    const auto encryptedKey =
        RsaOperation::EncryptText(sessionKey_, publicKey);
    SendFrame(PacketType::kEncryptedDesKey,
              SerializeEncryptedKey(encryptedKey));
    des_.SetKey(sessionKey_);
    return true;
}
```

这一实现具有三点意义：

- 保留了第一次实验中已经完成的 DES 模块，无需重写；
- 将 RSA 仅用于密钥分发，符合混合加密的典型设计思路；
- 自动生成会话密钥和自动共享会话密钥真正落地到程序运行过程。

### 4. 分帧传输与 DES 加密聊天

由于同一条 TCP 连接上既要传输 RSA 公钥和密钥分发报文，又要传输 DES 密文，因此 Lab2 在 Lab1 的长度头机制基础上进一步加入了**类型字段**。具体实现中，程序先发送 1 字节报文类型，再发送 4 字节网络字节序长度，最后发送有效载荷。

```
bool Chat::SendFrame(PacketType type,
                     const std::vector<std::uint8_t>& payload) {
    const std::uint8_t typeByte = static_cast<std::uint8_t>(type);
    const std::uint32_t payloadLength =
        static_cast<std::uint32_t>(payload.size());
    const std::uint32_t networkLength = htonl(payloadLength);
    SendAll(&typeByte, sizeof(typeByte));
    SendAll(reinterpret_cast<const std::uint8_t*>(&networkLength),
            sizeof(networkLength));
    return payloadLength == 0 || SendAll(payload.data(), payload.size());
}

bool Chat::ReceiveFrame(PacketType& type,
                        std::vector<std::uint8_t>& payload) {
    std::uint8_t typeByte = 0;
    RecvAll(&typeByte, sizeof(typeByte));
    type = static_cast<PacketType>(typeByte);

    std::uint32_t networkLength = 0;
    RecvAll(reinterpret_cast<std::uint8_t*>(&networkLength),
            sizeof(networkLength));
    const std::uint32_t payloadLength = ntohl(networkLength);
    payload.assign(payloadLength, 0);
    return payloadLength == 0 || RecvAll(payload.data(), payload.size());
}
```

完成握手后，聊天阶段仍继续复用 Lab1 中已经实现的 DES 加密逻辑。也就是说，Lab2 并没有替换掉 DES 聊天模块，而是在其前面增加了一层 RSA 驱动的密钥分发流程。保留旧功能、增加新能力实现本次实验升级。

### 5. 聊天交互与退出控制

为了让程序更适合命令行交互和实验展示，当前实现对提示符、退出控制和输入清洗都做了增强。主线程发送循环的关键逻辑如下。

```
while (isRunning_) {
    std::cout << '[' << selfName << "]>␣" << std::flush;
    if (!std::getline(std::cin, message)) {
        isRunning_ = false;
        break;
    }

    message = NormalizeUserInput(message);

    if (message == kExitCommand) {
        std::cout << selfName << "␣is␣closing␣the␣connection.\n";
        isRunning_ = false;
        break;
    }

    const auto cipherText = des_.EncryptString(message);
    if (config_.showCipherHex) {
        std::cout << "[cipher␣hex]␣" << ToHex(cipherText) << '\n';
    }
    SendFrame(PacketType::kChatCipherText, cipherText);
}
```

这一部分主要体现了以下升级点：

- 终端提示符能够明确显示当前身份，如 `[Server]>` 或 `[Client]>`；
- 输入 `quit` 时，本端直接关闭连接，而不会将其当作聊天消息发送；
- 发送前对输入进行归一化处理，以降低删除键、控制字符和部分终端输入残留对显示结果的影响；
- 若开启 `show-cipher` 选项，程序会额外输出十六进制密文，便于展示加密效果。

# 五、 实验结果和分析

## 1. 编译与可执行文件结果

程序能够在 WSL 的 Linux 环境下通过 CMake 成功构建,生成可执行文件 `bin/RSA_DES_chat`。构建命令如下。

```
cmake -S . -B build
cmake --build build -j
```

## 2. 功能测试结果

| 测试项目 | 测试内容 | 结果 |
|---|---|---|
| CMake 构建测试 | 在 WSL 中执行 `cmake -S . -B build` 与 `cmake --build build -j` | 通过 |
| RSA 握手测试 | 服务器端发送公钥，客户端加密回传 DES 密钥，服务器端成功解密 | 通过 |
| DES 密钥自动分发 | 双方无需手动输入密钥即可建立安全通道 | 通过 |
| 英文聊天测试 | 双方发送英文消息并正确加密、解密与显示 | 通过 |
| 中文聊天测试 | 双方发送 UTF-8 中文消息并正确显示 | 通过 |
| 密文显示测试 | 启用 `show-cipher` 后能显示十六进制密文 | 通过 |
| 报文分帧测试 | 握手与聊天报文均能稳定解析 | 通过 |
| 退出控制测试 | 输入 `quit` 后本端直接退出，对端感知断连 | 通过 |

## 3. 结果分析

1. **Lab2 的核心升级已经实现**。与第一次实验相比，程序最重要的改进是由手工输入 DES 密钥改为客户端自动生成会话密钥、服务器端自动接收并恢复，实现了真正的密钥自动分配。

2. **混合加密结构合理**。 RSA 负责小数据量的密钥传输，DES 负责连续聊天消息加密，既利用了 RSA 的密钥管理优势，也保留了 DES 对连续数据处理效率较高的特点。

3. **通信流程更加完整**。当前程序已经能够在一条 TCP 连接上依次完成建连、公钥发送、会话密钥回传、DES 加密聊天和退出控制，整体逻辑较为严密。

4. **程序可演示性较强**。身份提示、密文显示选项、握手完成提示和本地退出命令都使程序更加适合实验展示。

5. **仍存在实验性质的局限**。当前 RSA 实现采用较小素数和逐字节加密方式，主要用于课程实验；此外程序仍是单客户端模型，Socket 收发方式也仍以线程配合阻塞 I/O 为主，与实际工程中的成熟安全协议相比还有较大差距。

## 4. 程序运行截图

为了更加直观地展示程序运行效果，这里补充三类截图：程序启动与 RSA 握手成功界面、客户端与服务器端互相发送英文消息和中文消息并显示密文，以及输入 `quit` 后的连接断开界面。

图 3 展示服务端与客户端启动后的连接过程，Current identity Role RSA handshake completed Secure channel is ready 等关键输出说明程序已经完成 TCP 建连和会话密钥自动分配。

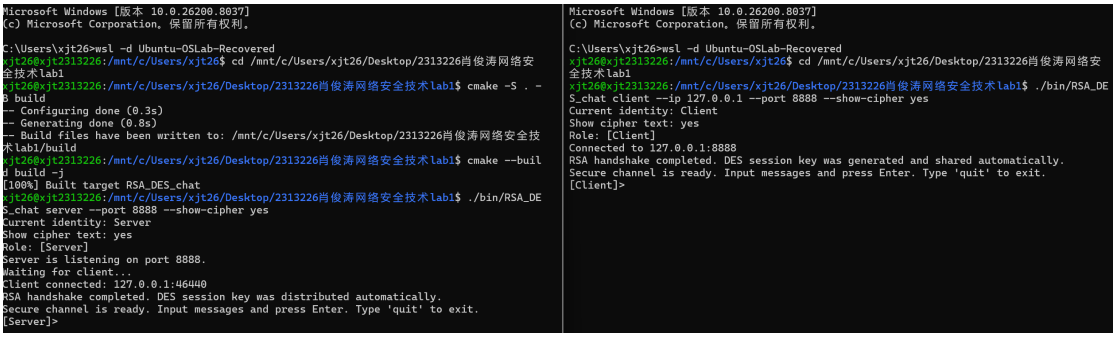



图 3: 程序启动与 RSA 握手界面

图 4 建议展示客户端和服务器端相互发送英文和中文消息的效果。该图应体现客户端输入、服务端收到消息以及密文显示三部分内容，用于证明 DES 加密聊天已经在自动分配的会话密钥上正常工作。同时即使消息内容为 UTF-8 中文文本，程序仍能正确完成加密、传输、解密和显示，验证了 Lab2 对中英文输入的兼容性。

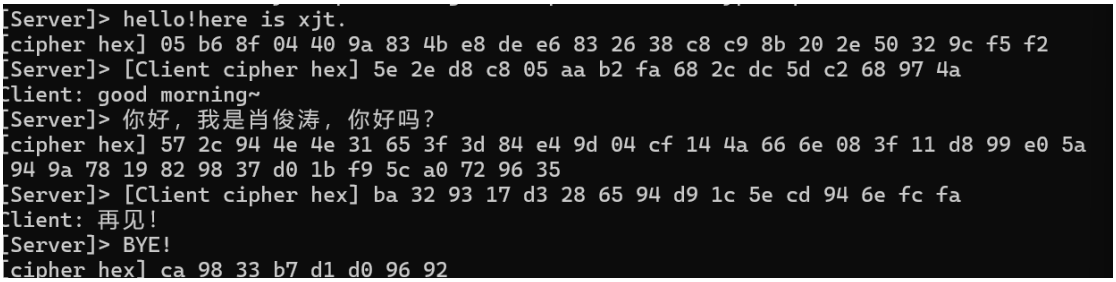



图 4: 聊天界面

图 5 展示输入 `quit` 后的退出效果。能够同时体现本端关闭连接和对端检测到断开连接两个现象，从而说明程序的退出控制逻辑符合预期。

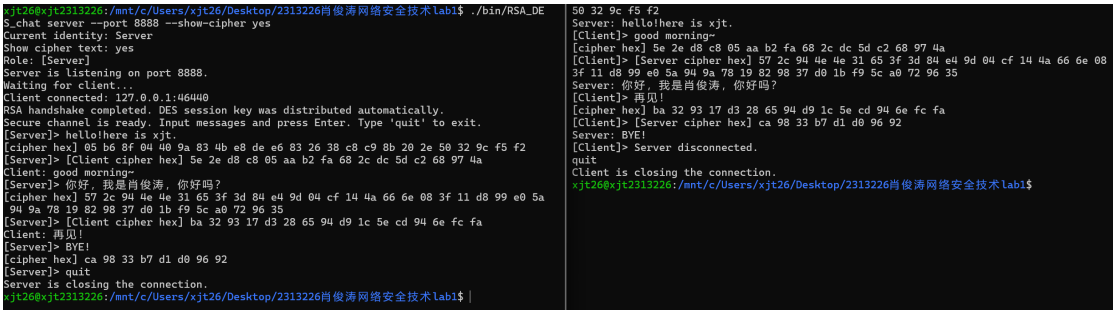



图 5: 连接控制和退出（总体概览）

# 六、 收获与反思

通过本次实验，我对**对称加密、非对称加密与网络通信三者之间的协同关系**有了更加系统的理解。第一次实验主要解决的是“如何用 DES 对聊天消息进行加密”，而第二次实验进一步解决了“DES 密钥如何安全地分发”这个真实通信系统的问题。将 RSA 与 DES 结合之后，我对混合加密结构的工程意义有了更直观的认识。

此外，本实验也让我进一步体会到协议设计的重要性。仅仅实现加密算法本身并不足以支撑一个完整的通信程序，还必须考虑 TCP 字节流的分帧问题、握手报文类型区分、线程收发模型以及命令行交互细节。只有把这些环节都处理清楚，程序的逻辑才能真正闭合。

当然，当前程序仍然主要面向课程实验。后续若继续完善，可以从以下方向改进：一是使用更标准、更安全的现代密码算法；二是引入更规范的密钥交换和报文认证机制；三是采用 `select/epoll` 等 I/O 模型改进并发能力；四是扩展为多客户端通信系统。